Arquitectura del proyecto `BibliotecaClean`

Refactorización de `model/service/ui` hacia Clean Architecture

Material para Ingeniería de Software

Marzo de 2026

Propósito de la refactorización

- Separar claramente **dominio**, **casos de uso**, **adaptadores** e **infraestructura**.
- Hacer que las **dependencias apunten hacia adentro**.
- Evitar que la interfaz de usuario concentre lógica de negocio.
- Facilitar futuros cambios, por ejemplo:
  - cambiar consola por interfaz web,
  - sustituir almacenamiento en memoria por base de datos,
  - agregar nuevas reglas sin romper otras capas.

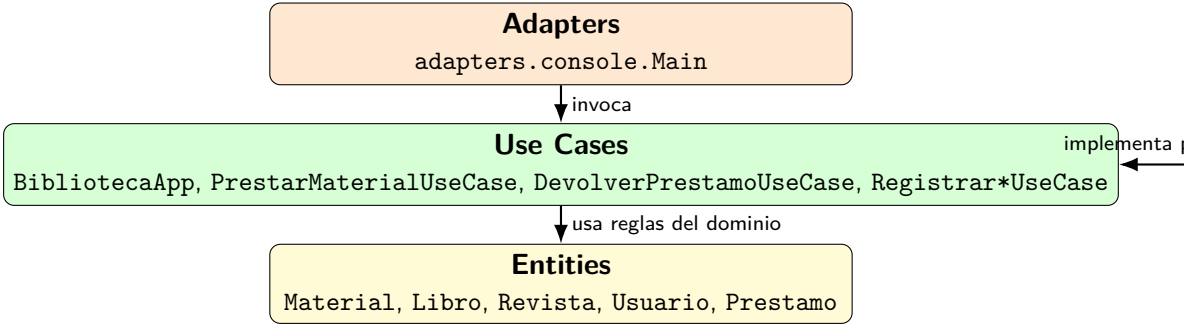
Estructura general del proyecto

```
BibliotecaClean/
|- src/
| |- entities/
| |- usecases/
| | |- dto/
| | |- ports/
| | \- services/
| |- infrastructure/
| | |- repositories/
| | \- services/
| \- adapters/
| \- console/
\- docs/
```

Idea central

Cada paquete responde a una responsabilidad bien definida y no mezcla detalles de interfaz con reglas del negocio.

Mapa arquitectónico



La dirección correcta de dependencias es: adaptadores e infraestructura dependen de casos de uso; los casos de uso dependen de entidades.

Capa `entities`

- Contiene el **modelo del dominio**.
- No depende de consola, repositorios ni detalles técnicos.
- Aquí están las clases:
  - `Material`
  - `Libro` y `Revista`
  - `Usuario`
  - `Prestamo`
  - `Prestable`

Responsabilidad

Representar el estado y el comportamiento esencial del sistema de biblioteca.

Ejemplos

- disponibilidad del material,
- cálculo de costo base,
- asociación entre usuario, material y préstamo.

Capa usecases

- Orquesta la lógica de aplicación.
- Aquí viven los casos de uso:
  - `RegistrarUsuarioUseCase`
  - `RegistrarLibroUseCase`
  - `RegistrarRevistaUseCase`
  - `PrestarMaterialUseCase`
  - `DevolverPrestamoUseCase`
- `BibliotecaApp` actúa como **fachada** para la UI.

Subpaquetes importantes

- `ports`: interfaces como `MaterialRepository`, `UsuarioRepository`, `PrestamoRepository`, `IdGenerator`
- `dto`: `OperationResult`
- `services`: implementación de los casos de uso

Puertos + implementaciones

Puerto en `usecases.ports`

```java
public interface MaterialRepository {
    void save(Material material);
    Material findById(String id);
    List<Material> findAll();
}
```

Implementación en `infrastructure`

```java
public class InMemoryMaterialRepository
        implements MaterialRepository {
    private final List<Material> materiales
        = new ArrayList<>();
    // save, findById, findAll
}
```

Ventaja

La lógica del préstamo no conoce cómo se almacenan los datos; solo conoce el contrato.

Capa `infrastructure`

- Aporta los **detalles concretos** del sistema.
- En este proyecto incluye:
  - `InMemoryMaterialRepository`
  - `InMemoryUsuarioRepository`
  - `InMemoryPrestamoRepository`
  - `SimpleIdGenerator`
- Esta capa puede cambiar sin reescribir las reglas del negocio.

Ejemplo de evolución

Hoy el sistema usa listas en memoria. Mañana podría usar MySQL, PostgreSQL o MongoDB manteniendo los mismos puertos.

Capa `adapters.console`

- Contiene la interfaz de usuario por consola.
- La clase `Main`:
  - muestra el menú,
  - lee entradas con `Scanner`,
  - crea `BibliotecaApp` e inyecta implementaciones concretas,
  - imprime el resultado de cada operación.
- No contiene reglas de préstamo; solo **coordina interacción humana**.

Inyección simple de dependencias

La consola instancia repositorios e ID generator y los pasa a `BibliotecaApp`.

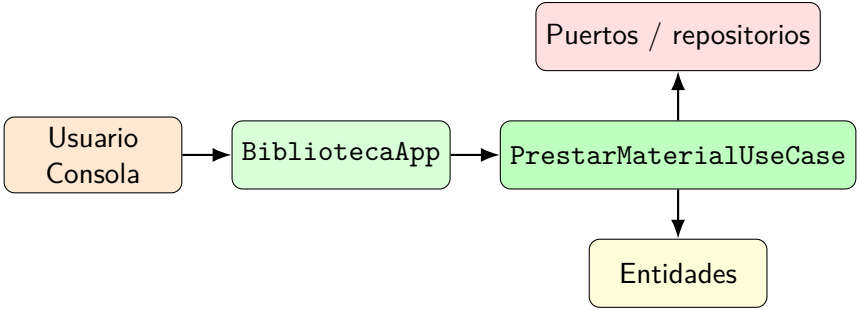
Caso de uso clave: PrestarMaterialUseCase

```java
public OperationResult ejecutar(String idUsuario,
        String idMaterial, int dias) {
    Usuario usuario = usuarioRepository.findById(idUsuario);
    Material material = materialRepository.findById(idMaterial);

    if (usuario == null) return fail("Usuario no encontrado");
    if (material == null) return fail("Material no encontrado");
    if (!material.estaDisponible()) return fail("No disponible");
    if (dias > ReglasPrestamo.DIAS_MAX_PRESTAMO) return fail(...);

    Prestamo prestamo = new Prestamo(idGenerator.nextId('P'),
            usuario, material, dias);
    prestamoRepository.save(prestamo);
    material.prestar();
    return ok(...);
}
```

Flujo del préstamo



1. La UI captura IDs y días.
2. `BibliotecaApp` delega al caso de uso.
3. El caso de uso consulta puertos, valida reglas y crea el préstamo.
4. El resultado se devuelve como `OperationResult` para que la UI lo muestre.

Antes vs. después de la refactorización

Proyecto original

- paquetes `model/service/ui`
- `Biblioteca` concentraba mucha lógica
- acoplamiento directo entre UI y servicios concretos
- menos preparado para nuevas interfaces o persistencia real

Proyecto refactorizado

- capas `entities/usecases/infrastructure/a`
- casos de uso separados por responsabilidad
- puertos para repositorios e IDs
- UI más delgada y dominio más estable

La mejora principal no es el nombre de los paquetes, sino la separación de dependencias y responsabilidades.

Conclusión

Resumen

`BibliotecaClean` muestra una evolución clara desde una arquitectura académica simple hacia una arquitectura más limpia, desacoplada y extensible.

- **Entities**: representan el dominio.
- **Use cases**: coordinan la lógica de aplicación.
- **Infrastructure**: implementa detalles concretos.
- **Adapters**: conecta el sistema con el usuario.

Regla final: las dependencias solo apuntan hacia adentro.